Relatório da Aula 007 - 02/10/2026

Slide 04:

Minhas estimativas correspondem a estas:

1) Python: Entendo que o operador "**" seja relativo a potência, então Considero que ele tenha a prioridade

R: 4, 8, 9

2) Java: Considero que será exibido 339. Entendo que ele fará a soma primeiro e a concatenação depois

3) JavaScript: Não consigo determinar desse resultado, pois eu Entendo que JavaScript possibilita operações de números com strings, então não sei o que pensar

4) C: Nessa situação também Não consigo determinar do que será exibido como resultado

5) Python: Não consigo determinar do que será exibido aqui

6) JavaScript: Considero que o valor obtido será

false, false, false

Os gabaritos são:

1) -4512 -> No primeiro trecho, encontramos -2 ** 2. Você poderia pensar que seria +4, contudo em Python a precedência do "**" é maior do que o "-" unário. Em outras palavras, ele faz a operação de 2 elevado a 2 e depois deixa o "-" no valor, portanto, -4. Em relação ao 512, eu me confundi no meu palpite.
A razão é que ele eleva 2 ** 3 ** 2, Em outras palavras é tudo junto, não separado como tinha imaginado. Para o Python, ele
interpreta da esquerda para direita a ordem das potências, Em outras palavras:

2 elevado a 3, este sendo elevado a 2 => 2 ** (3 ** 2) => 512

2) 3345 -> No primeiro trecho da expressão (1 + 2), ele realizará soma, produzindo 3.
Em seguida, apresenta (+ "3"). Nessa situação, o + é empregado para concatenar, produzindo "33".
Depois, como passamos a concatenar strings, os outros + vão seguir a mesma lógica (+ 4 + 5).
Resultando em
"3345"

3) 352 -> No primeiro trecho, encontramos uma subtração ("5" - 2). Como o - é empregado para números, ele executa a subtração
como se fosse um número, gerando 3. No segundo trecho encontramos o + ("5" + 2). Nessa situação, ele trata como
concatenação, produzindo "52". Ao final, encontramos "352"

4) 0 -> Aqui a lógica está como Verdadeiro (1) e Falso (0). 3 > 2 é verdadeiro, logo dá 1.
Isso leva a próxima parte: 1 > 1, que é falso, resultando no 0 que recebemos no terminal.

5) padrão True -> Comportamento observado é que 0 em Python é falso por padrão. Logo, em "x or padrão", quem
sai é "padrão). No segundo trecho, encontramos 3 > 2 > 1, o qual o Python interpreta como 3 > 2 and 2 > 1, que gera
True. Então Ao final encontramos "padrão True"

6) true false true -> Na primeira comparação, o == converte a string "" em número. Ele vira 0, então a comparação passa a ser
0 == 0, o que é true. Na segunda comparação, ele compara duas strings distintos, dá false. Na terceira é um caso clássico de
operador de igualdade em JS. Mesmo um sendo string e o outro número, ele não distingue os tipos, então dá true.

Slide 05:

Os tipos de precedência das operações dentro da linguagem podem variar.

Pense na expressão a + b * c, como exemplo.

Podemos fazer de distintos formas:

1) Da esquerda para direita: (a + b) * c

2) Da direita para esquerda: a + (b * c)

Dependendo da expressão, posso ter resultados distintos pela precedência.

Ex: a = 3, b = 4, c = 5

1) (3 + 4) * 5 = 35

2) 3 + (4 * 5) = 23

Slide 06:

Temos diferentes expressões

1) -2 ** 2
2) 2 ** 3 ** 2
3) 1 + 2 + "3"
4) -7 / 2
5) -7 % 2

Vamos como cada linguagem lida com essas contas.

Phyton:

1) -4 -> Precedência do ** sobre o unário
2) 512
3) Erro -> Não aceita int + str
4) -4 -> com //, para divisão de inteiros
5) 1

JavaScript:

1) Erro -> Ele não usar ** com número negativo
2) 512
3) "33" -> Primeiro ele realiza a soma dos números e depois concatena
4) -3.5
5) -1

Java:

1) Erro -> não existe ** em Java
2) Erro -> mesma razão
3) "33" -> Primeiro ele soma enquanto apresenta só números. Quando entram strings, ele concatena.
4) -3 -> Ele acaba truncando a divisão para baixo
5) -1

O slide também trata de Associatividade. Esse conceito fala sobre que direção uma expressão com operadores de mesma prioridade é calculada.

Como exemplo, em várias linguagens, * e / apresenta a mesma prioridade, então pela associatividade, saberemos em qual direção a expressão será executada

ESQUERDA -> DIREITA

10 / 2 * 3

Agora pensando em **, no Python, a associatividade muda. Aí é da direita para a esquerda:

2 ** 3 ** 2 -> primeiro 3 ** 2 e depois 2 ** 9

Slide 07:

Este slide apresenta efeitos colaterais que funções podem exercer em expressões, dependendo da linguagem.

No exemplo, encontramos esse código aqui:

```
int a = 5;

int fun1(void) {
   a = 17;       /* efeito colateral */
   return 3;
}

int main(void) {
   a = a + fun1();   /* 8 ou 20? */
   printf("a = %d\n", a);
}
```

Dependendo da linguagem, o valor de a pode ser alterado para 17 na execução da função, produzindo 20 Ao final OU
o a pode manter o seu valor 5 Ao final gerar 8. Depende se a função apresenta a capacidade de gerar esse efeito colateral ou não

EM JAVA: O resultado é 8, pois ele realmente vai ler a expressão da esquerda para direita. Então "a" manterá o valor 8.

EM C: O resultado vai ser 20, pois não há uma ordem definida por padrão como Java.

EM GO: O resultado dá 20, assim como C

Nos dois últimos casos, observamos que a função foi capaz de exercer um efeito colateral na expressão.

Slide 08:

Este slide apresenta a sobrecarga de operadores.

Sobrecarga de operadores é quando um mesmo operador pode ter distintos significados dependendo dos tipos dos valores envolvidos.

Como exemplo, o operador + pode somar números (2 + 3), concatenar strings ("Olá" + "mundo") ou até
representar uma operação entre objetos, como vetores ou matrizes,
em linguagens que permitem definir esse comportamento.

Esse comportamento pode deixar o código mais natural e legível, pois podemos escrever v1 + v2 em vez de chamar uma função específica para somar dois vetores.

Contudo, a sobrecarga também pode dificultar a compreensão.

A sobrecarga pode ser definida pela própria linguagem ou pelo programador.

Slide 09:

Este slide aborda conversões de tipos e os distintos comportamentos.

Ocorrem acontecem quando um valor de um tipo é transformado em outro.

No primeiro caso encontramos o alargamento, Em outras palavras, um tipo menor é convertido para um tipo maior, como int para float,
sem grande risco de perda de informação.

EX: Em Java

```
int grande = 123456789;
float f = grande;
```

No segundo, o estreitamento transforma um tipo maior em um menor e pode provocar perda de dados, sendo muitas vezes necessário fazer um cast explícito.

Ex: Em Java

```
double x = 3.99;

int y = (int) x;

System.out.println(y); //3
```

Já a coerção ocorre quando a própria linguagem converte tipos de modo automático para permitir uma operação.

Ex: Em Java

```
int x = 5;
double y = 2.5;

double resultado = x + y;

System.out.println(resultado); //7.5
```

—----------

Slide 10:

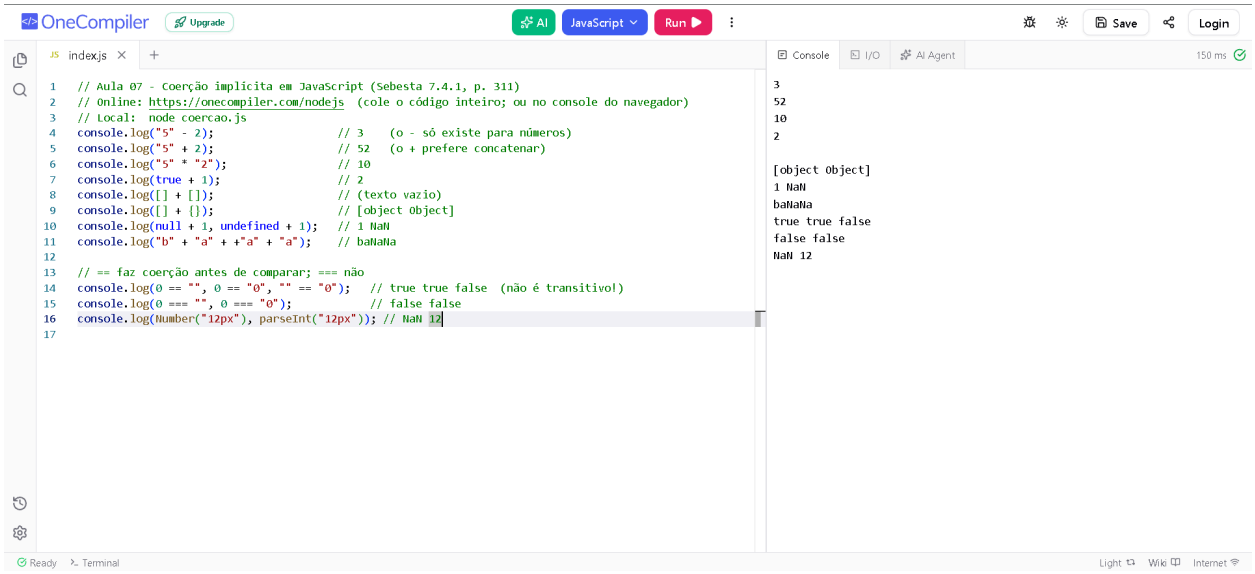
Coerção implícita: É quando o interpretador ou o compilador faz a troca de um tipo de uma variável em outra, sem que isso seja solicitado anteriormente. Isso em geral ocorre em linguagens pouco tipadas. Como no slide de exemplo sendo o Javascript, um grande exemplo. Em geral o PHP não é considerado uma linguagem fortemente tipada, mas tanto o PHP quanto o Javascript podem ocorrer a tipagem. No conteúdo do slide encontramos uma atividade para compararmos dois tipos de igualdades distintos, vamos começar com o “===” do JavaScript, dentro da linguagem ele significa igualdade estrita, Em outras palavras, a comparação não vai ser somente por aquilo que o dado representa, mas também pelo seu tipo, então vamos igualar o valor e seu tipo, isso nos dá uma segurança contra a coerção implícita que é feita pela linguagem. Agora o “!==”, pela linguagem PHP, ela significa o não identico, Em outras palavras, ela não é igual ao “!=”, que compara somente o valor, nela a comparação é igual ao “===”, que falei anteriormente, ela compara tanto o tipo quanto o valor, servindo como uma barreira de proteção para a coerção implicita. Podemos constatar a partir dos exemplos que encontramos aqui uma escolha da linguagem, ou trade-off, onde essas linguagens escolheram a facilidade de escrita e velocidade em vez da confiabilidade.

Comentários do slide:

“"b" + "a" + +"a" + "a" = "baNaNa": o +"a" é o mais unário aplicado a "a", que dá NaN. Regra prática em JavaScript e PHP: use === e !== (não fazem coerção). Linters como o ESLint (regra eqeqeq) obrigam isso.
PHP (type juggling): "0e123" == "0e456" é true porque os dois textos parecem números em notação científica e valem 0. Os hashes MD5 de "240610708" e "QNKCDZO" começam com "0e" seguidos só de dígitos. Um login que fizesse md5($senha) == $hash_salvo aceitaria uma senha pela outra. O PHP 8 corrigiu parte disso: "abc" == 0 passou a ser false, mas "0e..." == "0e..." continua true.
Ligação com o Sebesta: o custo da coerção é a perda de detecção de erros (p. 312). Linguagens de script escolheram facilidade de escrita em vez de confiabilidade.”

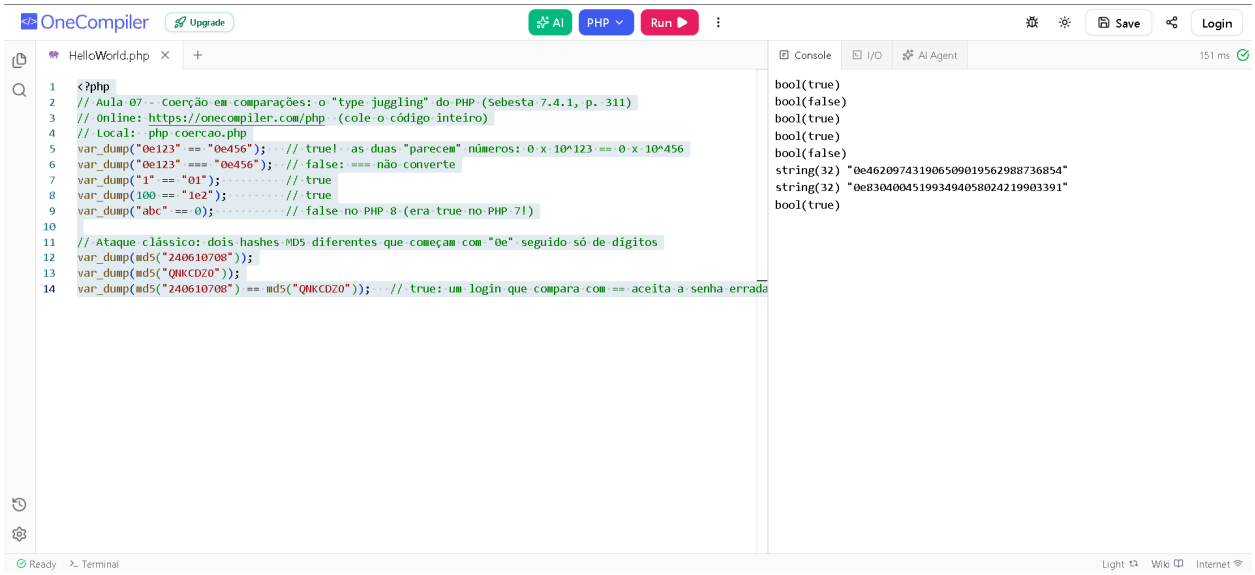
Demonstração:

JavaScript:



Observações: Nesta compilação conseguimos observar claramente a parte do implícito, ou, de como a linguagem julga os tipos.

Php:



Observações: Nesta compilação da linguagem de php conseguimos verificar prontamente como a linguagem realiza a coerção implícita, e é possível observar claramente o “Type Juggling”, onde os dois valores “Parecem iguais” ao PHP por começarem com a mesma letra e número.

Slide 11:

Expressões relacionais e booleanas: Expressões relacionais e booleanas: É a forma como as linguagens avaliam e comparam valores lógicos e matemáticos, e como a ordem dessas operações afeta o desfecho. Um ponto relevante é a precedência: os operadores relacionais, como o de maior ou menor, têm uma precedência menor que os aritméticos. Ou seja, em uma expressão como "a + 1 > 2 * b", a linguagem primeiro resolve a matemática para só depois comparar os resultados. No conteúdo do slide encontramos uma atividade de quiz para

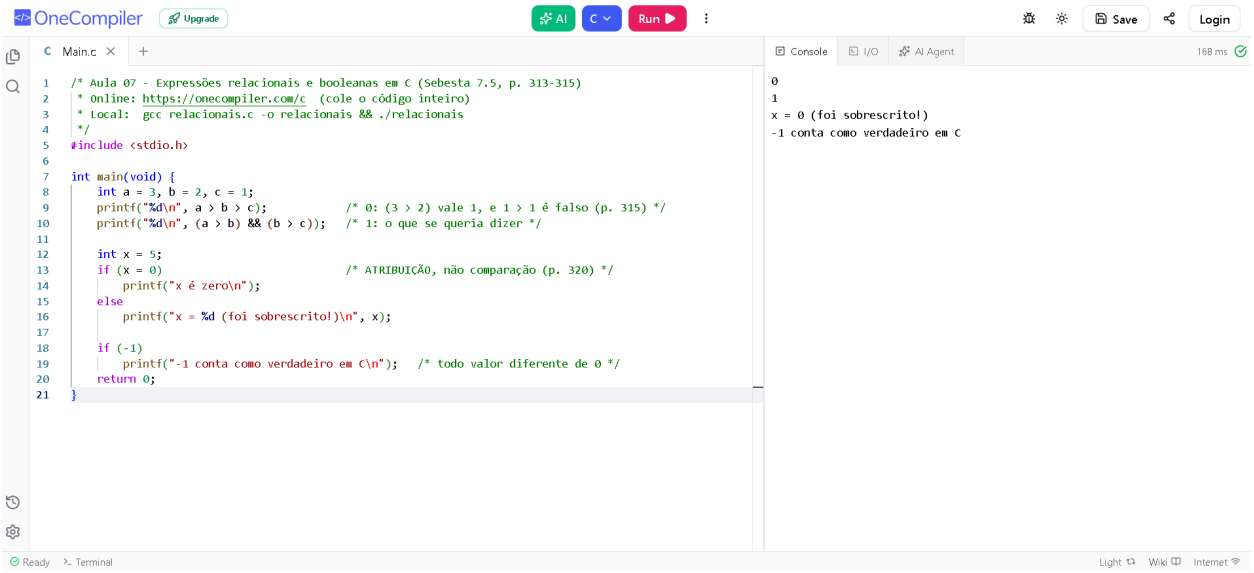
compararmos como distintos linguagens lidam com a mesma expressão: "3 > 2 > 1". Em C, essa expressão nos dá um resultado incomum e falso. A razão é que o C avalia a conta da esquerda para a direita: ele primeiro faz "3 > 2", o que é verdadeiro e resulta no número 1; em seguida, ele compara esse "1 > 1", o que se torna falso. Por outro lado, em Python, essa mesma expressão é encadeada de modo automático pela linguagem, significando "(3 > 2) and (2 > 1)", o que resulta em verdadeiro, igual fazemos na matemática escolar. Adicionalmente, encontramos o conceito de "Truthiness", onde linguagens sem um controle rígido aceitam qualquer valor numa condição. Em Python, um 0 ou uma lista vazia contam como falso, mas uma string "0" ou uma lista com itens contam como verdadeiro. Linguagens modernas como Java ou C# exigem estritamente um booleano (true ou false) nessas verificações. Podemos constatar Com base nesses exemplos mais uma escolha da linguagem, ou trade-off, onde o Python e o C clássico escolheram a flexibilidade e facilidade de escrita, enquanto Java e C# escolheram restringir isso em prol da confiabilidade e detecção precoce de erros.

Observações sobre o slide: Tempo: ~3 min. DEMONSTRAÇÃO: relacionais.c e relacionais.py.

Sebesta (p. 313-314): os operadores relacionais têm precedência MENOR que os aritméticos (a + 1 > 2 * b faz a aritmética primeiro). O resultado é booleano, exceto em C89, que não tinha tipo booleano: 0 é falso e qualquer outro valor é verdadeiro (p. 315).
Resposta do quiz: em C, 3 > 2 > 1 vale 0. O livro chama isso de "resultado incomum" (p. 315): (3 > 2) dá 1, e 1 > 1 é falso. A comparação entre b e c nunca acontece.
Em Python, a mesma expressão é ENCADEADA: 3 > 2 > 1 significa (3 > 2) and (2 > 1), que é True, como na matemática.
"Truthiness": Python e JavaScript aceitam qualquer valor numa condição. 0, "", [], {} e None contam como falso, mas "0", [0] e " " contam como verdadeiro. Java, C#, Go e Rust EXIGEM um booleano no if, e o livro (p. 315) defende isso pela legibilidade e pela detecção de erros.
Perl e Ruby têm && e 'and' com precedências distintos (p. 315).

Demonstração

C:



```c
/* Aula 07 - Expressões relacionais e booleanas em C (Sebesta 7.5, p. 313-315)
 * Online: https://onecompiler.com/c  (cole o código inteiro)
 * Local:  gcc relacionais.c -o relacionais && ./relacionais
 */
#include <stdio.h>

int main(void) {
    int a = 3, b = 2, c = 1;
    printf("%d\n", a > b > c);            /* 0: (3 > 2) vale 1, e 1 > 1 é falso (p. 315) */
    printf("%d\n", (a > b) && (b > c));   /* 1: o que se queria dizer */

    int x = 5;
    if (x = 0)                            /* ATRIBUIÇÃO, não comparação (p. 320) */
        printf("x é zero\n");
    else
        printf("x = %d (foi sobrescrito!)\n", x);

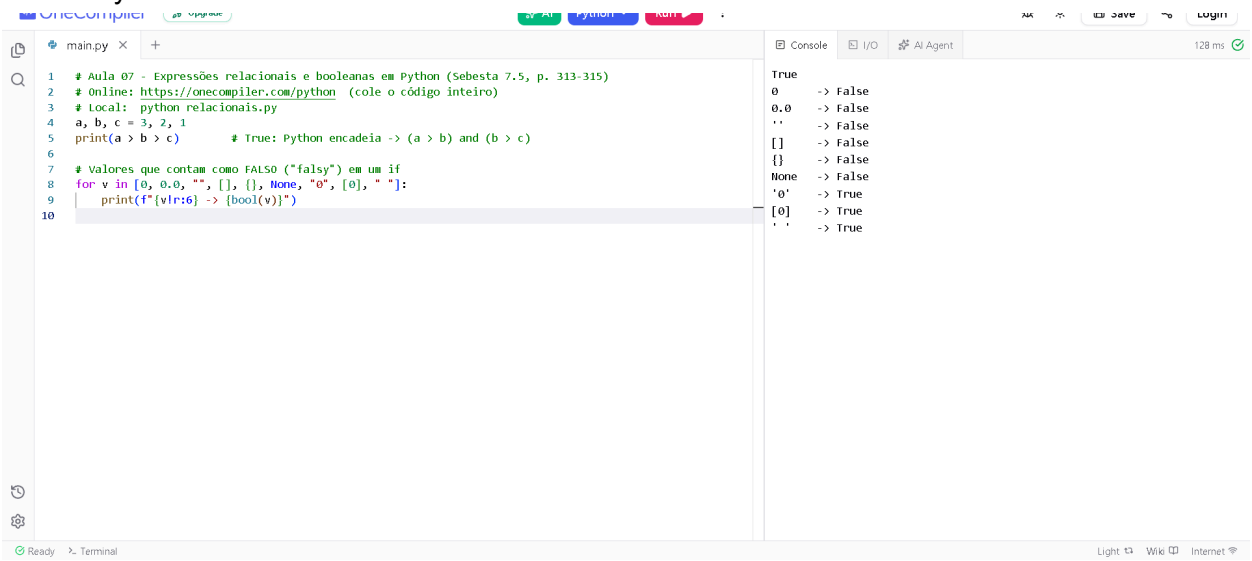
    if (-1)
        printf("-1 conta como verdadeiro em C\n");   /* todo valor diferente de 0 */
    return 0;
}
```

```
0
1
x = 0 (foi sobrescrito!)
-1 conta como verdadeiro em C
```

Comentários:

Precedência: O compilador resolve primeiro as expressões matemáticas (a + 1 e 2 * b) para só depois aplicar o operador relacional (>), retornando 1 (representação de verdadeiro em C). O "resultado incomum": A expressão 3 > 2 > 1 imprime 0 (Falso). Isso ocorre porque a linguagem C avalia da esquerda para a direita. Primeiro ela resolve (3 > 2), que retorna o número 1. Em seguida, ela faz 1 > 1, o que é matematicamente falso, produzindo 0. A variável c nem precisaria existir, a lógica de encadeamento falha. Tipagem (Falta de booleano estrito): Como o C89 não possuía o tipo bool nativo, qualquer número diferente de 0 (como o -42 do exemplo) é aceito pelo if como verdadeiro, o que pode provocar bugs difíceis de rastrear em comparação com linguagens mais modernas (como Java e Rust, que barrariam esse código).

Python:



```python
# Aula 07 - Expressões relacionais e booleanas em Python (Sebesta 7.5, p. 313-315)
# Online: https://onecompiler.com/python  (cole o código inteiro)
# Local:  python relacionais.py
a, b, c = 3, 2, 1
print(a > b > c)        # True: Python encadeia -> (a > b) and (b > c)

# Valores que contam como FALSO ("falsy") em um if
for v in [0, 0.0, "", [], {}, None, "0", [0], " "]:
    print(f"{v!r:6} -> {bool(v)}")
```

```
True
0      -> False
0.0    -> False
''     -> False
[]     -> False
{}     -> False
None   -> False
'0'    -> True
[0]    -> True
' '    -> True
```

Comentários:

Encadeamento natural: Diferente do C, Python trata 3 > 2 > 1 de maneira natural e matemática. Internamente, o interpretador traduz isso para (3 > 2) and (2 > 1), o que resulta de modo correto em True. Isso aumenta drasticamente a legibilidade (Capacidade de Leitura) da linguagem. Truthiness: Python é muito flexível com o que considera verdadeiro ou falso (truthiness). O código demonstra que o if aceita strings e listas. O comportamento padrão é: coleções ou strings vazias ("", []) correspondem a consideradas False. Porém, se houver qualquer conteúdo dentro delas — até mesmo uma string com o número zero ("0") ou uma lista com o zero dentro ([0]) —, Python avaliará a condição como True.

Slide 12:

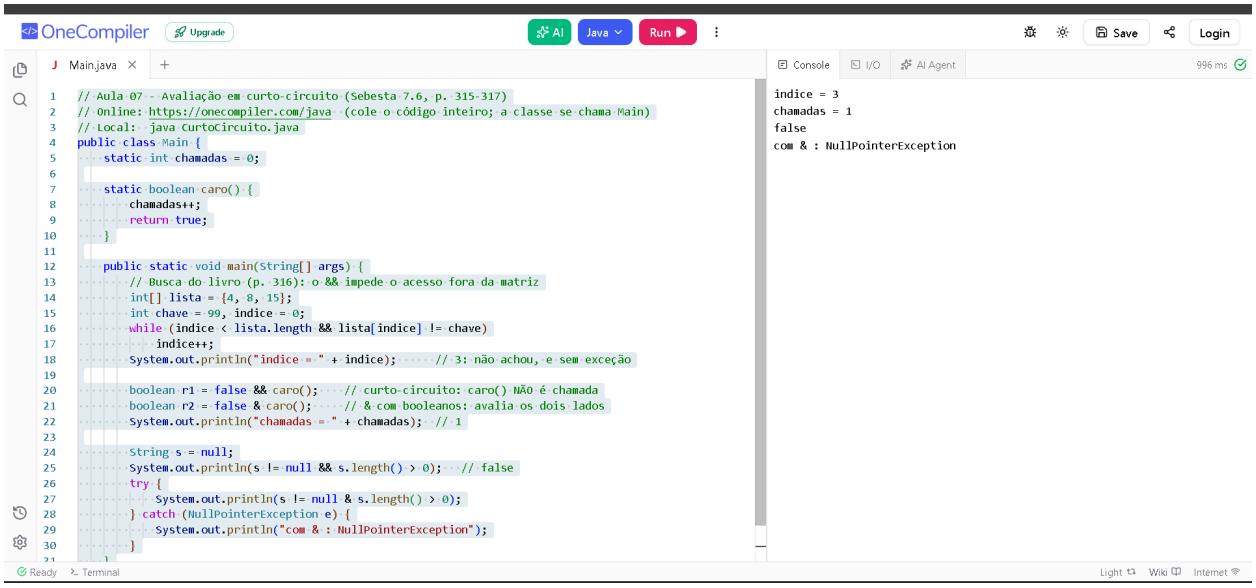
Avaliação em curto-circuito:É quando o resultado de uma expressão lógica é determinado sem a necessidade de avaliar todos os seus operandos. Como exemplo, em uma expressão como (a >= 0) && (b < 10), se a < 0, a primeira parte já é falsa. Como é uma operação "E" (&&), o desfecho será falso independentemente do valor de b. Portanto, a linguagem nem sequer avalia o segundo termo. Isso não apenas otimiza o código, mas também serve como um mecanismo de segurança essencial. Imagine uma busca em um array: while ((index < listlen) && (list[index] != key)). Se o índice ultrapassar o tamanho da lista, o primeiro termo falha, acionando o curto-circuito. Se a linguagem não fizesse isso, ela tentaria acessar list[listlen], o que geraria um erro fatal de índice fora dos limites. O risco do curto-circuito ocorre quando o segundo operando apresenta um efeito colateral, como um incremento (b++). Se o curto-circuito for acionado, o b++ nunca acontecerá. Linguagens baseadas em C utilizam && e || para curto-circuito, enquanto operadores como & e | (sem repetição) avaliam ambos os lados, mesmo com booleanos. Linguagens como Python, Ruby e Perl aplicam curto-circuito em todos os operadores lógicos. Modernamente, linguagens adotaram curto-circuitos especializados para lidar com nulos, como o encadeamento opcional (?.) e a coalescência nula (??). Um bug frequente em JavaScript (e Python) ocorre ao usar o operador || (ou or) para definir valores padrão: a expressão 0 || 10 retornará 10, pois o 0 é avaliado como falso (truthiness), sobrescrevendo um valor que poderia ser válido. O operador ?? resolve isso focando exclusivamente em valores nulos.

Observações sobre o slide: “Tempo: ~4 min. DEMONSTRAÇÃO: CurtoCircuito.java e depois curto_circuito.js.
Definição (p. 315): o resultado é determinado sem avaliar todos os operandos. Em (a >= 0) && (b < 10), se a < 0 o segundo termo é irrelevante.
Exemplo do livro (p. 316): a busca while ((index < listlen) && (list[index] != key)). SEM curto-circuito, quando a chave não está na lista, list[listlen] seria acessado e lançaria uma exceção de índice. O curto-circuito protege o acesso.
Linguagens baseadas em C: && e || fazem curto-circuito. & e | com booleanos NÃO fazem (p. 317). Ruby, Perl, ML, F# e Python fazem curto-circuito em todos os operadores lógicos (p. 317).
Risco (p. 316): curto-circuito combinado com efeito colateral. Em (a > b) || ((b++) / 3), o b++ só acontece às vezes.
Modernos: o ?. (encadeamento opcional) e o ?? (coalescência nula) de JavaScript, TypeScript, C# e Kotlin correspondem a curto-circuitos especializados em null.

Resposta do quiz (5): em Python, x or "padrão" retorna "padrão" quando x = 0. O or retorna um dos OPERANDOS, e o 0 conta como falso. Por esse motivo quantidade || 10 é um bug em JS quando 0 é válido, e o ?? resolve.”

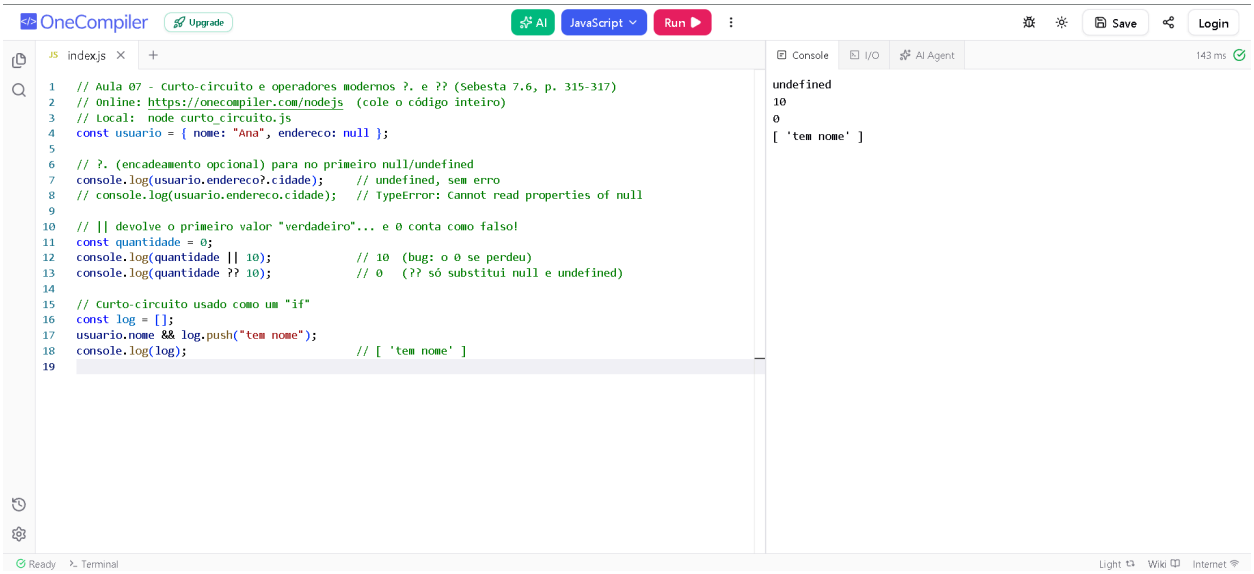
Demonstração:

Java:



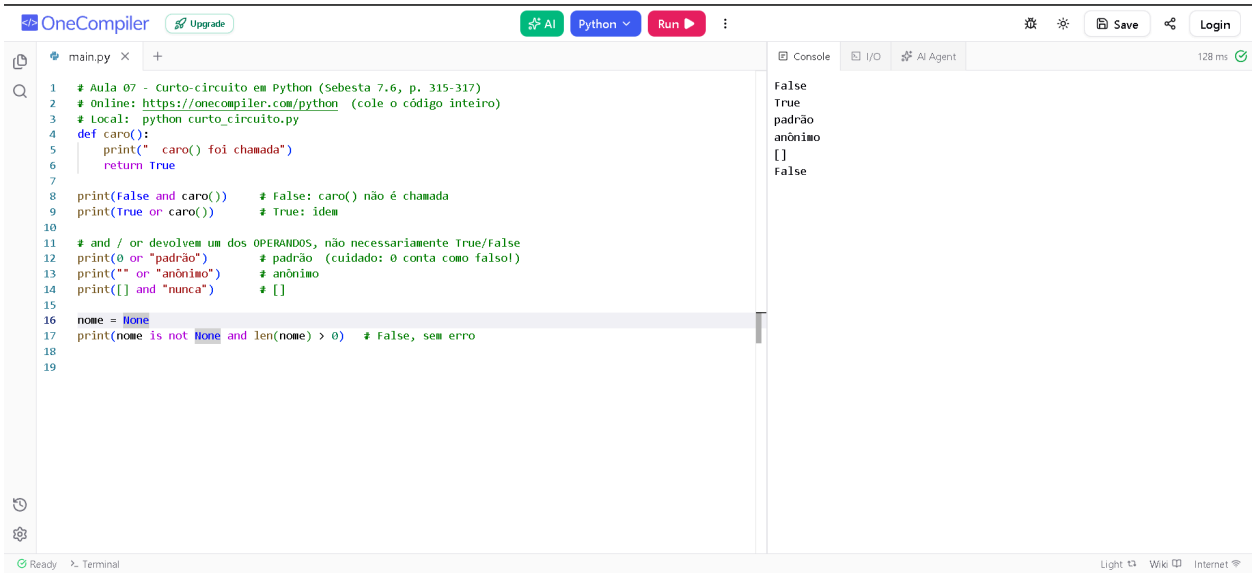
Observações: Neste exemplo prático em Java, é possível observar como o operador && protege contra exceções. No laço while, ele evita um erro de índice. O exemplo com a função caro() evidencia a distinção crucial: && faz o curto-circuito, interrompendo a avaliação no false, enquanto o & comercial avalia os dois lados incondicionalmente, mesmo que a primeira condição já determine o resultado. O último bloco prova como a falta do curto-circuito (&) causa um NullPointerException ao tentar checar o comprimento de uma string nula.

JavaScript:



Observações: O código JavaScript ilustra evoluções recentes para lidar com ausência de dados de maneira segura. O operador ?. faz um "curto-circuito" ao encontrar uma propriedade nula, evitando um TypeError que paralisaria o sistema. O código também demonstra claramente a falha ao usar || para valores padrão: se a quantidade for legitimamente zero, o || a descarta, assumindo o valor 10. O novo operador ?? soluciona isso. Por fim, vemos um truque frequente: o uso do && como um "if" em uma única linha.

Python:



Observações:Em Python, todos os operadores lógicos (and, or) possuem curto-circuito nativo. Como provado na função caro(), o segundo operando é completamente ignorado se a primeira parte da expressão já garantir o resultado. O mais relevante é que and e or não devolvem valores booleanos estritos (True ou False), mas sim o último operando avaliado, demonstrando que 0 or "padrão" retorna o texto "padrão", o que pode provocar o mesmo bug exibido no JavaScript. O último exemplo mostra o curto-circuito atuando de maneira idêntica ao Java para proteger uma variável nula (None) de causar erros durante verificações.

Slide 13:

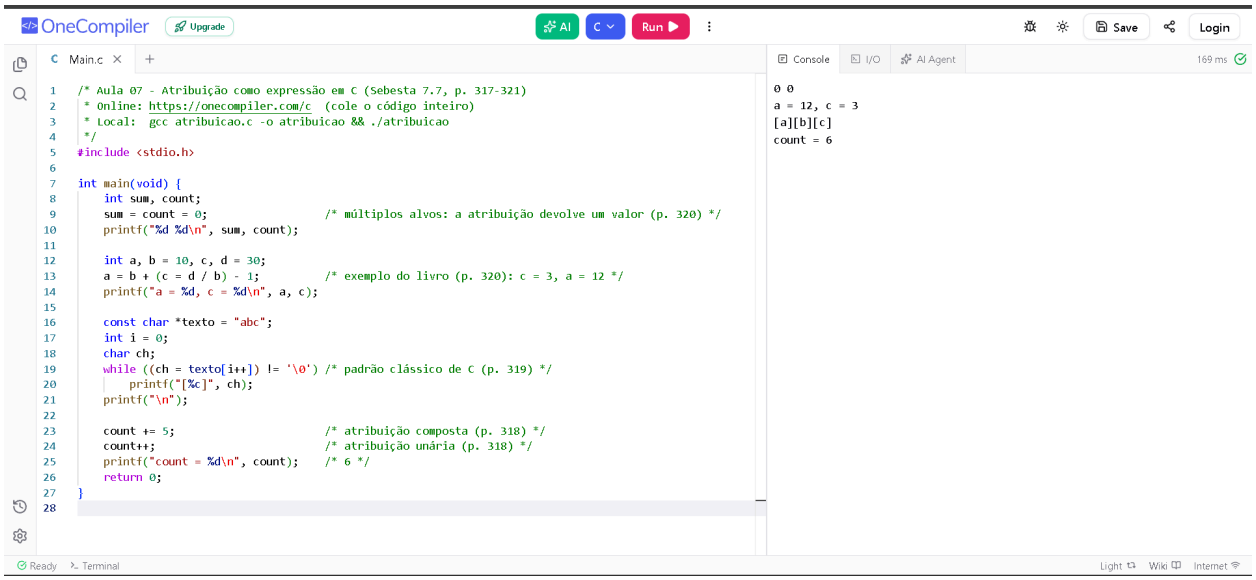
Sentenças de atribuição: A atribuição é a operação mais essencial em linguagens imperativas, responsável por alterar o estado de uma variável armazenada na memória. A forma simples mais adotada pelas linguagens é o =, embora o ALGOL utilizasse := justamente para evitar confusão com o operador matemático de igualdade. Além da forma simples, encontramos a atribuição composta (como a += b, herdada do ALGOL 68 e popularizada pelo C) e a atribuição unária (count++), que condensa a operação em um único operando. Um conceito muito relevante discutido pelo Sebesta é a atribuição agindo como uma EXPRESSÃO. Em linguagens como C, JavaScript e Perl, quando você atribui um valor (a = 5), essa operação inteira retorna o próprio valor 5. Isso possibilita encadeamentos do tipo sum = count = 0 (lido da direita para a esquerda) ou construções complexas dentro de laços while. O custo dessa flexibilidade é a confiabilidade: em C, é muito frequente cometer o erro de digitar if (x = y) em vez

de if (x == y); o compilador aceita, a atribuição ocorre no meio do if, e a condição é avaliada com base no valor atribuído, causando bugs terríveis. Linguagens mais seguras, como Java e C#, bloquearam isso exigindo que o if receba estritamente um valor booleano. Por outro lado, linguagens modernas como Python e Go trouxeram a "atribuição múltipla", permitindo substituir valores de duas variáveis com um simples a, b = b, a, sem precisar de uma terceira variável temporária. O Python, historicamente, proibiu que a atribuição fosse uma expressão para evitar o bug do if (x = y). No entanto, na versão 3.8, foi introduzido o "operador morsa" (:=), permitindo atribuir e testar o valor na mesma linha, de maneira controlada.

Observações sobre o slide: “Tempo: ~5 min. DEMONSTRAÇÃO: atribuicao.c, atribuicao.py e atribuicao.go.
Formas (p. 317-321):
- Simples: quase todas as linguagens usam =. ALGOL usou := para não confundir com a igualdade.
- Composta: a += b (vem do ALGOL 68 e foi adotada por C).
- Unária: count++ e ++count. Em - count ++, o incremento acontece antes da negação (p. 319).
- Como EXPRESSÃO (p. 319-320): em C, JavaScript e Perl, a atribuição retorna o valor atribuído. Isso possibilita while ((ch = getchar()) != EOF) e sum = count = 0. O exemplo do livro a = b + (c = d / b) - 1 aparece no arquivo.
- O custo (p. 320): if (x = y) em vez de if (x == y) compila em C. Java e C# evitam o problema porque o if só aceita booleano.
- Múltipla (p. 320-321): Perl, Ruby, Lua, Python e Go permitem a, b = b, a (troca sem variável temporária).
- Funcional (p. 321): em ML e F#, val e let dão NOME a um valor, que nunca muda. O let de Rust é imutável por padrão, e "let x = x + 1" cria uma NOVA variável (sombreamento).
Python: a atribuição NÃO é expressão, exceto pelo operador morsa := (3.8), criado justamente para casos como o while do getchar. Go: x++ é sentença, e y := x++ não compila.”

Demostração:

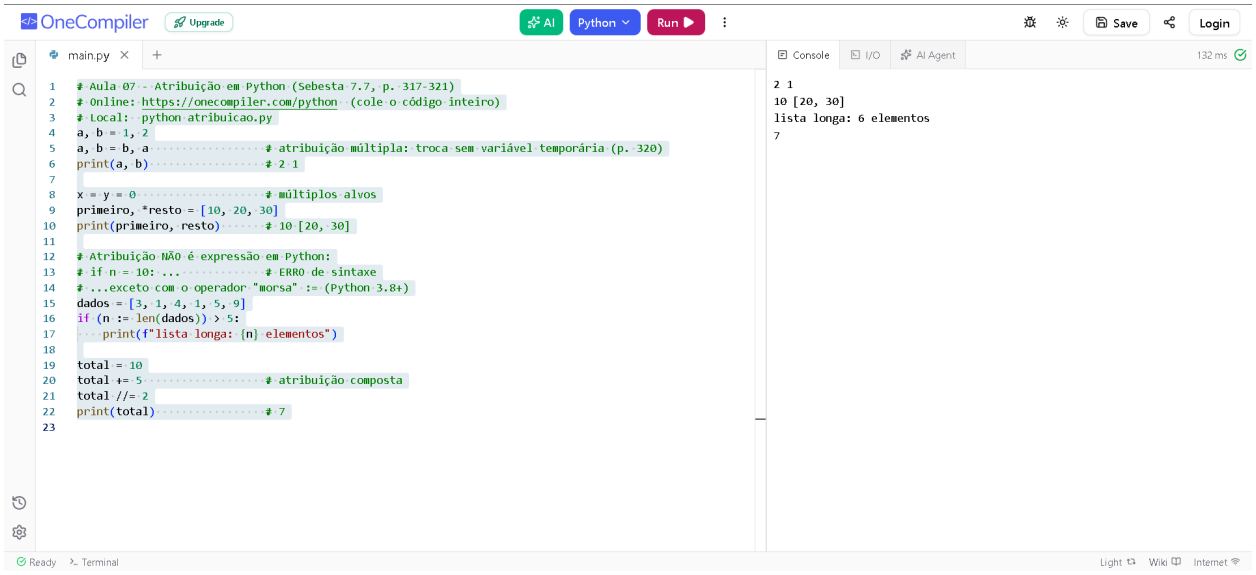
C:



Observações:O código em C ilustra perfeitamente a atribuição funcionando como

uma expressão que retorna valor. Isso é evidenciado na linha sum = count = 0, onde a avaliação da direita para a esquerda atribui 0 a count, que por sua vez "retorna" 0 para ser atribuído a sum. O "exemplo do livro" (a = b + (c = d / b) - 1) mostra como Esse comportamento pode deixar o código terrivelmente confuso (baixa legibilidade) ao embutir uma atribuição matemática no meio de outra expressão. Por fim, vemos o padrão clássico do C no laço while, onde a variável ch recebe o caractere e, simultaneamente, o while avalia se esse caractere recém-atribuído é diferente do fim da string ('\0').

Python:



Observações: O script em Python destaca a conveniência da "atribuição múltipla", permitindo inverter valores na mesma linha (a, b = b, a) de maneira muito limpa e legível. Mostra também o recurso de desempacotamento de listas usando o asterisco (*resto). O ponto crucial dessa demonstração é a proteção do Python contra a atribuição acidental em condições: tentar usar dentro de um if gera um erro de sintaxe imediato. Para contornar isso e ainda permitir atribuições atreladas a verificações lógicas, o Python exige o uso do operador morsa (:=), como visto no teste do comprimento da lista (n := len(dados)).

Slide 14:

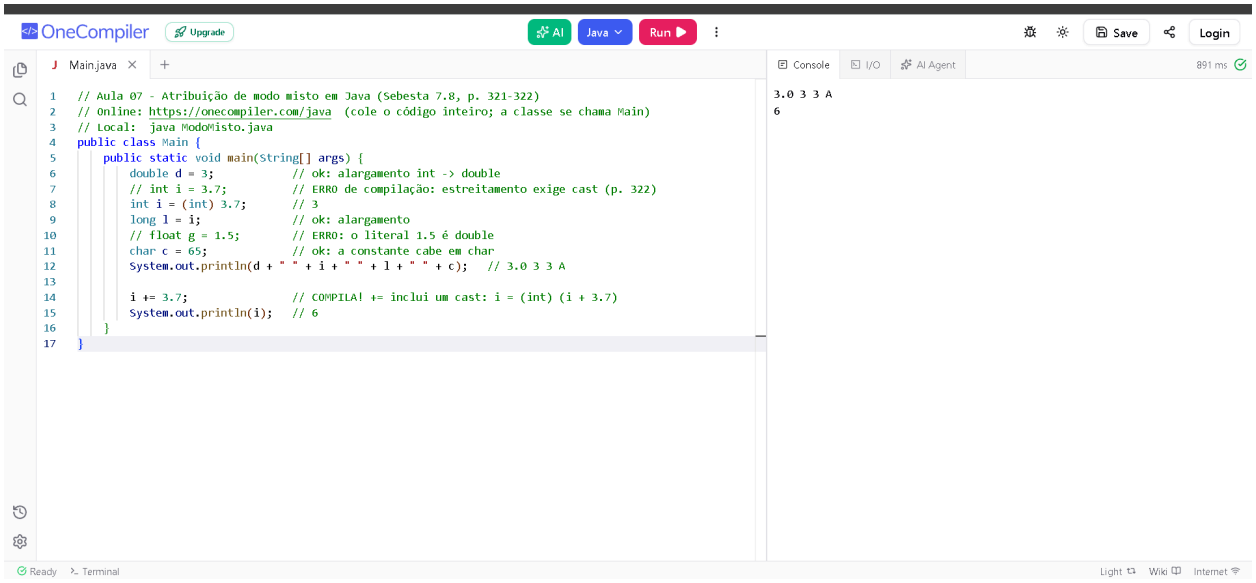
Atribuição de modo misto: Atribuição de modo misto: Ocorre quando atribuímos um valor de um tipo a uma variável de outro tipo. A grande questão de projeto discutida pelo Sebesta é: a expressão precisa ter exatamente o mesmo tipo de variável, ou a linguagem deve forçar uma coerção (conversão automática)? Linguagens mais antigas como C, C++ e Perl permitem a coerção em qualquer direção, na maioria das vezes de maneira totalmente silenciosa. Isso significa que você pode colocar um número quebrado (3.7) dentro de um inteiro, e a linguagem simplesmente corta a parte decimal sem avisar. Java e C# decidiram aumentar a confiabilidade proibindo metade dessas conversões: elas só permitem a coerção de ALARGAMENTO, Em outras palavras, você pode colocar um int dentro de um double (pois não há perda de dados), mas o contrário gera um erro de compilação imediato. Em linguagens puramente funcionais, não existe atribuição de modo misto de maneira alguma. Porém, há uma pegadinha clássica em Java com atribuições compostas: se você fizer int i = 3; e depois i += 3.7;, o código vai compilar perfeitamente! Isso se deve ao fato de que a especificação do Java define que

operadores como += possuem um cast implícito embutido. Conectando com nossos exercícios anteriores: ao fazermos double taxa = acertos / total * 100, se acertos e total forem inteiros, a divisão será puramente inteira (7 / 10 = 0), e só DEPOIS esse 0 será alargado para 0.0 para caber no double, gerando um erro lógico.

Observação do slide: “Tempo: ~3 min. DEMONSTRAÇÃO: ModoMisto.java e modo_misto.c.
Questão de projeto (p. 321): a expressão precisa ter o mesmo tipo da variável que recebe o valor, ou vale coerção?
- C, C++ e Perl: coerção em qualquer direção. int i = 3.7 vira 3 em silêncio, e char c = 300 vira 44 (o gcc só avisar).
- Java e C# (p. 322): só permitem a coerção de ALARGAMENTO (int para float, mas não o contrário). Segundo o livro, proibir metade das atribuições de modo misto é um jeito simples e eficaz de aumentar a confiabilidade.
- Linguagens funcionais: não existe atribuição de modo misto.
Pegadinha do Java: i += 3.7 COMPILA, porque a especificação define que E1 op= E2 equivale a E1 = (T)((E1) op (E2)), com um cast implícito.
Conecte com o exercício (questão 6): double taxa = acertos / total * 100 dá 0.0. A divisão é INTEIRA (7 / 10 = 0) e só DEPOIS o resultado é alargado para double. ”

Demonstração

Java:



Observações: Nesta compilação em Java, verificamos a regra de proteção (alargamento) em ação: tentar colocar 3.7 direto no int foi barrado pelo compilador para evitar perda de dados, exigindo que o programador force a barra escrevendo o (int) de forma explícita. No entanto, o trecho final do código prova a pegadinha da linguagem: a operação composta i += 3.7 passa ilesa pela segurança do compilador porque embute secretamente o cast explícito que era exigido na atribuição simples, truncando silenciosamente a casa decimal e resultando em 6.

C:



Observações: A execução do código em C expõe o risco da conversão silenciosa em qualquer direção (estreitamento). A atribuição int i = 3.7; simplesmente joga a casa decimal no lixo sem dar nenhum alerta. O problema escala com char c = 300;: como um char suporta no máximo o valor 255, o número transborda (overflow) e o compilador gcc emite apenas um pequeno aviso não-bloqueante (warning), armazenando o lixo matemático 44 na variável, o que destruiria a confiabilidade do software em produção.

Slide 15:

Estrutura de controle: Realizando agora a transição do Capítulo 7 para o 8, saímos da simples atribuição de valores para o fluxo do programa em si. Como o livro de Sebesta aponta, computações imperativas avaliam e atribuem, mas programas úteis precisam de dois mecanismos essenciais: escolher caminhos (seleção) e repetir ações (iteração). Como é possível observar na imagem, em 1966, Böhm e Jacopini provaram matematicamente que qualquer algoritmo existente pode ser escrito usando apenas três coisas: uma sequência, um "if-else" e um laço "while". Isso provou que o famoso e caótico "goto" era totalmente desnecessário, dando origem à programação estruturada. Mas se apenas três comandos bastam, por que linguagens modernas têm for, switch, do-while, foreach? A resposta é a facilidade de escrita. Criar estruturas específicas poupa tempo do desenvolvedor. No entanto, o limite para essa criação de estruturas é a legibilidade: se uma linguagem se torna gigante, os programadores acabam aprendendo subconjuntos distintos dela, tornando o código de um ilegível para o outro. Por fim, definimos uma estrutura de controle como a sentença em si (o "if") mais o bloco que ela controla. A regra de ouro do design moderno é: as estruturas possuem apenas uma porta de entrada. Múltiplas entradas exigiram o retorno do famigerado "goto" e seus rótulos, destruindo a organização lógica do código.

Observação do slide: “Tempo: ~2 min. Transição do Cap. 7 para o Cap. 8.

Sebesta (p. 328): as computações imperativas avaliam expressões e atribuem valores, mas programas úteis precisam de mais dois mecanismos: SELECIONAR entre caminhos e REPETIR sentenças.

Böhm e Jacopini (1966) provaram que qualquer algoritmo representável em fluxograma pode ser escrito só com sequência, seleção de dois caminhos e laço com pré-teste. O goto é desnecessário. Esse resultado apoiou a programação estruturada.

Por que as linguagens têm tantas estruturas a mais (for, switch, do-while, foreach...)? Pela facilidade de escrita (p. 329). O limite é a legibilidade: poucas pessoas aprendem todas as sentenças de uma linguagem grande, e cada leitor conhece um subconjunto diferente.

Definição (p. 330): uma estrutura de controle é uma sentença de controle mais as sentenças cuja execução ela controla. Questão de projeto geral: a estrutura pode ter MÚLTIPLAS ENTRADAS? (Hoje, uma entrada só, porque múltiplas entradas exigiram goto e rótulos.)”

Demonstração:

Comparação Conceitual: Linguagens Ricas (C/Java) vs. Linguagens Minimalistas (Go)

C: Linguagens como C, C++, Java e C# oferecem um arsenal vasto de estruturas de controle. Se você quer repetir algo, pode usar "while", "do-while", o "for" clássico (com inicialização, condição e incremento) ou o "foreach" (para iterar sobre coleções). Isso dá ao programador a ferramenta perfeita para cada micro-situação, maximizando a velocidade e a facilidade de escrita. Porém, como Sebesta alerta, isso exige que quem lê o código saiba exatamente como cada uma dessas estruturas se comporta (como exemplo, lembrar que o "do-while" executa pelo menos uma vez antes de testar a condição).

Go: A linguagem Go, criada pelo Google, levou a teoria de Böhm e Jacopini ao pé da letra para resolver o problema de "subconjuntos de linguagem" citado pelo Sebesta. Em Go, NÃO EXISTE "while", "do-while" ou "foreach". Existe apenas uma única estrutura de repetição: o "for". Se você quer um loop infinito, escreve apenas "for {}". Se quer um while, escreva "for (condição)". Se quer o clássico, escreve "for i:=0; i<10; i++". Os projetistas da linguagem sacrificaram propositalmente a criação de palavras-chave novas (reduzindo a facilidade de escrita) para garantir 100% de legibilidade: qualquer programador Go no mundo lê e escreve laços exatamente da mesma forma.

Slide 16:

Else pendente: É quando existem comandos de seleção aninhados com mais "if" do que "else", gerando a dúvida de a qual "if" o "else" pertence. Em C, a regra é que o "else" pertence ao "if" mais próximo que ainda não apresenta "else", e a endentação é ignorada pelo compilador, o que pode enganar o programador, como No exemplo apresentado, que imprime 1 em vez de -1. Para associar o "else" ao "if" externo, é preciso usar chaves. Por outro lado, em Python não existe essa ambiguidade, pois a endentação faz parte da sintaxe e define a qual "if" o "else" pertence. Em Rust o problema também não ocorre, porque as chaves correspondem a obrigatórias em todo "if" e "else".

Seletores que devolvem valor: Correspondem a seleções que produzem um valor que pode ser atribuído de modo direto a uma variável. Em C, o "if" é apenas um comando, então é usado o operador ternário (?:). Em Python existe a expressão condicional (valor if condição else valor). Por outro lado, em Rust o próprio "if" é uma expressão, Em outras palavras, o bloco selecionado retorna um valor, sendo obrigatório o uso do "else" e os dois ramos devendo ter o mesmo tipo.

Demonstração:

C:

Main.c

```c
#include <stdio.h>

int main(void) {
    int sum = 0, count = 5, result = -1;

    if (sum == 0)
        if (count == 0)
            result = 0;
    else
        result = 1;
    printf("Sem chaves: %d\n", result);      /* 1 */

    result = -1;
    if (sum == 0) {
        if (count == 0)
            result = 0;
    } else {
        result = 1;
    }
    printf("Com chaves: %d\n", result);      /* -1 */

    int x = -3;
    int y = x > 0 ? x : 2 * x;
    printf("Operador ternario: %d\n", y);    /* -6 */
    return 0;
}
```

Console | I/O | AI Agent

```
Sem chaves: 1
Com chaves: -1
Operador ternario: -6
```

Observação: Esta execução demonstra que o primeiro trecho, sem chaves, imprime 1, mostrando que o "else" foi associado ao "if" mais próximo, mesmo com a endentação sugerindo o contrário. Com as chaves, o "else" passa a pertencer ao "if" externo e o resultado é -1. Na última linha, o operador ternário retorna -6, mostrando como C faz uma seleção que produz valor.

Python:

main.py

```python
sum_, count, result = 0, 5, -1

if sum_ == 0:
    if count == 0:
        result = 0
else:
    result = 1
print("Else do if externo:", result)   # -1

result = -1
if sum_ == 0:
    if count == 0:
        result = 0
    else:
        result = 1
print("Else do if interno:", result)   # 1

x = -3
y = x if x > 0 else 2 * x
print("Expressao condicional:", y)     # -6
```

Console | I/O | AI Agent

```
Else do if externo: -1
Else do if interno: 1
Expressao condicional: -6
```

Observação: Esta execução demonstra que a endentação define a qual "if" o "else" pertence: alinhado ao "if" externo, o resultado é -1, e alinhado ao interno, é 1, sem nenhuma ambiguidade. A expressão condicional retorna -6, o mesmo valor obtido em C com o operador ternário.

Rust:




```rust
fn main() {
    let sum = 0;
    let count = 5;
    let mut result = -1;

    if sum == 0 {
        if count == 0 {
            result = 0;
        }
    } else {
        result = 1;
    }
    println!("Else do if externo: {}", result);   // -1

    let x = -3;
    let y = if x > 0 { x } else { 2 * x };
    println!("if como expressao: {}", y);         // -6
}
```

Observação: Durante esta execução o resultado é -1, pois as chaves obrigatórias deixam explícito que o "else" pertence ao "if" externo, eliminando o problema. Na segunda linha, o próprio "if" é usado como expressão e retorna -6 de modo direto para a variável y.

Slide 17:

Seleção múltipla: É quando o programa seleciona um entre vários caminhos de acordo com o valor de uma variável. Em C, o "switch" trata cada "case" apenas como um ponto de entrada, e a execução continua pelos casos seguintes até encontrar um "break", o que é chamado de "cair" (fallthrough). No exemplo apresentado, com index igual a 1, correspondem a executados odd, even e a mensagem de erro, pois não há nenhum "break". Esse comportamento possibilita agrupar valores, mas, por ser o padrão, o esquecimento de um "break" gera erros silenciosos.

As linguagens mais recentes corrigiram esse problema de formas distintos. Em Go, cada caso se encerra sozinho, e para cair no próximo é preciso escrever "fallthrough" de forma explícita. Em C#, cair implicitamente é proibido e gera erro de compilação. No Java 14 em diante, o "switch" com seta (->) não cai, possibilita agrupar valores com vírgula e pode ser usado como expressão, devolvendo um valor; Adicionalmente, é exaustivo para enum, Em outras palavras, o compilador acusa erro se algum valor não for tratado. Em Rust, o "match" compara o valor com padrões, como valores exatos, faixas (1..=9) e o curinga "*", que funciona como o "default". O "match" também é* exaustivo: o compilador verifica se todos os valores possíveis estão cobertos, e Por esse motivo, sem o *"*", o exemplo do slide não compila. O Python 3.10 também ganhou o "match", com padrões e guardas (condições extras em cada caso).

Evidências:

a) C (switch.c):

Main.c

```c
#include <stdio.h>

int main(void) {
    int index = 1, odd = 0, even = 0;

    switch (index) {
        case 1:
        case 3: odd += 1;
        case 2:
        case 4: even += 1;
        default: printf("Error\n");
    }
    printf("Sem break: odd=%d even=%d\n", odd, even);

    odd = 0; even = 0;
    switch (index) {
        case 1:
        case 3: odd += 1; break;
        case 2:
        case 4: even += 1; break;
        default: printf("Error\n");
    }
    printf("Com break: odd=%d even=%d\n", odd, even);
    return 0;
}
```

Console | I/O | AI Agent

```
Error
Sem break: odd=1 even=1
Com break: odd=1 even=0
```

Observações: Esta execução demonstra o efeito do "cair": sem "break", o valor 1 incrementou odd, incrementou even e ainda imprimiu "Error". Com o "break", apenas odd foi incrementado, que era o comportamento esperado.

b) Java (Switch.java):

Main.java

```java
public class Switch {
    enum Dia { SEG, TER, QUA, QUI, SEX, SAB, DOM }

    public static void main(String[] args) {
        for (Dia dia : Dia.values()) {
            String tipo = switch (dia) {
                case SAB, DOM -> "fim de semana";
                case SEG, TER, QUA, QUI, SEX -> "dia util";
            };
            System.out.println(dia + ": " + tipo);
        }
    }
}
```

Console | I/O | AI Agent

```
SEG: dia util
TER: dia util
QUA: dia util
QUI: dia util
SEX: dia util
SAB: fim de semana
DOM: fim de semana
```

Observação: Durante esta execução o "switch" com seta classifica cada dia sem precisar de "break" e sem cair para os outros casos. O resultado do "switch" é atribuído de modo direto à variável tipo, mostrando o switch como expressão, e como todos os dias do enum foram tratados, não foi necessário o "default".

c) Go (switch.go):

main.go

```go
package main

import "fmt"

func main() {
    index := 1
    odd, even := 0, 0

    switch index {
    case 1, 3:
        odd++
    case 2, 4:
        even++
    default:
        fmt.Println("Error")
    }
    fmt.Printf("Sem fallthrough: odd=%d even=%d\n", odd, even)

    odd, even = 0, 0
    switch index {
    case 1, 3:
        odd++
        fallthrough
    case 2, 4:
        even++
    default:
        fmt.Println("Error")
    }
    fmt.Printf("Com fallthrough: odd=%d even=%d\n", odd, even)
}
```

Console | I/O | AI Agent

```
Sem fallthrough: odd=1 even=0
Com fallthrough: odd=1 even=1
```

Observação: No primeiro trecho, cada caso se encerra sozinho e apenas odd é incrementado, sem precisar de "break". Na segunda, o "fallthrough" escrito de forma explícita faz a execução cair no caso seguinte, incrementando também even, mas sem chegar ao "default". Isso evidencia que em Go cair é uma escolha do programador e vale só para o próximo caso.

d) Rust (selecao.rs):

HelloWorld.rs

```rust
fn main() {
    for n in [-7, 0, 5, 42] {
        let tipo = match n {
            i32::MIN..=-1 => "negativo",
            0 => "zero",
            1..=9 => "um digito",
            _ => "varios digitos",
        };
        println!("{}: {}", n, tipo);
    }
}
```

Console | I/O | AI Agent

```
-7: negativo
0: zero
5: um digito
42: varios digitos
```

Observação: Durante esta execução o "match" classifica cada número usando padrões: a faixa dos negativos, o valor exato 0, a faixa de 1 a 9 e o curinga "_" para o restante. Cada número entra em exatamente um caso, e o valor retornado pelo "match" é atribuído de modo direto à variável tipo.

Slide 18:

Laços controlados por contador: Correspondem a laços que repetem um bloco de código usando uma variável que conta as repetições. Em C, o "for" é formado por três partes (inicialização,

condição e incremento), mas na prática funciona como um "while", pois a condição é testada a cada volta com o valor atual da variável. Por esse motivo ele é chamado de laço lógico disfarçado. A variável de controle é uma variável frequente, e alterá-la dentro do corpo muda a contagem: no exemplo do slide, o "i += 2" faz o laço imprimir apenas 0 3 6 9. Adicionalmente, as três partes do "for" correspondem a opcionais, e um "for" só com a condição equivale a um "while".

Em Python, o "for" não dispõe de condição nem incremento, ele percorre um iterável, entregando o próximo valor a cada volta. O "range(0, 10)" gera a sequência de 0 a 9 uma única vez no início, e Por esse motivo alterar o "i" dentro do corpo não muda nada: na volta seguinte o valor é sobrescrito pelo próximo da sequência, e o resultado continua sendo de 0 a 9. Em relação às questões de projeto, em C a variável declarada no "for" só existe dentro do laço e a condição é avaliada a cada volta, enquanto em Python a variável continua existindo depois do laço com o último valor recebido e o "range" é avaliado apenas uma vez. Assim, o mesmo código produz resultados distintos nas duas linguagens.

Evidências:
a) C (laco_for.c):

Main.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 10; i++) {
        printf("%d ", i);
        i += 2;
    }
    printf("\n");

    int i = 1, soma = 0;
    for (; i <= 100; )
        soma += i++;
    printf("Soma de 1 a 100: %d\n", soma);
    return 0;
}
```

Console | I/O | AI Agent

```
0 3 6 9
Soma de 1 a 100: 5050
```

Observação: Esta execução demonstra que alterar o "i" dentro do corpo muda a contagem, imprimindo apenas 0 3 6 9, pois a condição é testada a cada volta com o valor atual da variável. A segunda parte mostra o "for" só com a condição, funcionando como um "while" e somando os números de 1 a 100.

b) Python (laco_for.py):

main.py

```python
for i in range(0, 10):
    print(i, end=" ")
    i += 2
print()
print("Valor de i depois do laco:", i)

for i in range(10, 0, -3):
    print(i, end=" ")
print()
```

Console | I/O | AI Agent

```
0 1 2 3 4 5 6 7 8 9
Valor de i depois do laco: 11
10 7 4 1
```

Observação: Durante esta execução o laço imprime de 0 a 9 em geral, evidenciando que o "i += 2" não afetou a iteração, já que o "for" entrega o próximo valor do "range" a cada volta. A linha seguinte evidencia que o "i" continua existindo depois do laço, com o valor 11, e a última mostra o "range" com passo negativo, gerando 10 7 4 1.

Slide 19:

Laços lógicos, saídas e iteradores: Os laços lógicos correspondem a controlados por uma condição, e não por um contador, e Por esse motivo precisam garantir que haja progresso a cada volta para que a condição se torne falsa em algum momento. Um exemplo real de falha nisso foi o bug do Zune, em 2008: quando a variável dias valia exatamente 366 em um ano bissexto, nenhum dos caminhos do laço alterava as variáveis, e o laço se repetia infinitamente, travando os aparelhos no último dia do ano. Também é relevante diferenciar os laços de pré-teste, como o "while", que testam a condição antes de executar o corpo, dos laços de pós-teste, como o "do-while", que executam o corpo ao menos uma vez antes de testar, o que é fácil de esquecer.

Em relação às saídas, um "break" frequente sai apenas do laço mais interno. O Java possibilita dar nomes (rótulos) aos laços, e com o "break" rotulado é possível sair de vários laços aninhados de uma vez, como No exemplo apresentado, que interrompe a busca na matriz ao encontrar o primeiro número negativo. Por outro lado, em relação aos iteradores, o JavaScript apresenta duas formas parecidas com comportamentos distintos: o "for...in" percorre as chaves (os índices), que correspondem a entregues como texto, e Por esse motivo a soma acaba concatenando e gera "0012"; enquanto o "for...of" percorre os valores, resultando de modo correto em 60.

Evidências:

a) C (zune.c):



Observação: Esta execução demonstra o bug do Zune reproduzido: com dias igual a 366 no ano de 2008, o laço não fez progresso, pois nenhuma variável foi alterada, e só terminou porque um contador de segurança detectou o laço infinito, mostrando os valores travados. Na segunda

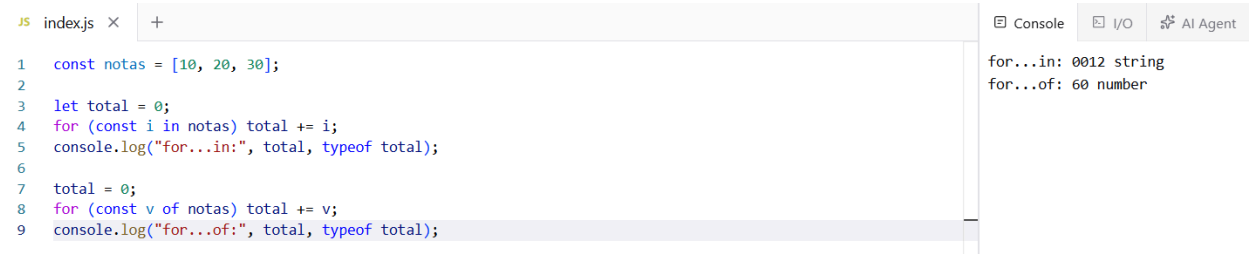
parte, o "while" não executou nenhuma vez, porque a condição já começava falsa, enquanto o "do-while" executou uma vez, mostrando a distinção entre pré-teste e pós-teste.

b) Java (Rotulos.java):



Observação: Durante esta execução, com o "break externo", o programa encontrou o primeiro negativo da matriz, na posição 0,2, e saiu dos dois laços, sem verificar a linha seguinte. Com o "break" frequente, ele saiu apenas do laço interno, continuou na linha 1 e encontrou também o negativo da posição 1,0, evidenciando que o rótulo muda de qual laço o programa sai.

c) JavaScript (iteracao.js):



Observação: Durante esta execução, o "for...in" percorreu os índices como texto, e a soma virou a concatenação "0012", do tipo string, por causa da coerção implícita. Já o "for...of" percorreu os valores do array, produzindo 60, do tipo number, que era o resultado esperado.

Slide 20:

Desvio incondicional: O "goto" é um comando que faz o programa saltar de modo direto para um ponto marcado com um rótulo, sem nenhuma condição. Em 1968, Dijkstra publicou a carta "Go To Statement Considered Harmful", argumentando que o "goto" torna o fluxo do programa difícil de acompanhar, o que impulsionou a programação estruturada. Hoje, C, C++, C# e Go ainda possuem "goto", enquanto Java não dispõe de, apesar de manter "goto" como

palavra reservada. Para os casos mais frequentes, as linguagens oferecem desvios restritos, como "break", "continue" e "return", que só podem ir para destinos bem definidos e Por esse motivo melhoram a legibilidade.

Um exemplo real dos riscos foi o bug "goto fail", encontrado no código de segurança da Apple em 2014. A função deveria verificar o hash e depois a assinatura do certificado, mas uma linha "goto fail" foi duplicada. Como o "if" não tinha chaves, apenas o primeiro "goto" pertencia a ele, e o segundo era executado sempre, apesar da endentação sugerir o contrário. Assim, quando o hash estava correto, a função pulava para o final com o valor 0, indicando "certificado válido", sem nunca verificar a assinatura, o que permitia aceitar certificados falsificados. O bug teria sido evitado com chaves obrigatórias, como em Go e Rust, ou compilando com "gcc -Wall", que avisa sobre endentação enganosa.

Evidências:

a) C (goto_fail.c):

Main.c

```c
#include <stdio.h>

int verifica_hash(void)       { return 0; }   /* hash correto */
int verifica_assinatura(void) { return -1; }  /* assinatura falsa */

int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
        goto fail;
    if ((err = verifica_assinatura()) != 0)
        goto fail;
fail:
    return err;
}

int valida_corrigido(void) {
    int err;
    if ((err = verifica_hash()) != 0) {
        goto fail;
    }
    if ((err = verifica_assinatura()) != 0) {
        goto fail;
    }
fail:
    return err;
}

int main(void) {
    int r1 = valida_certificado();
    int r2 = valida_corrigido();
    printf("Com o bug:  %d (%s)\n", r1, r1 == 0 ? "certificado valido" : "certificado invalido");
    printf("Corrigido: %d (%s)\n", r2, r2 == 0 ? "certificado valido" : "certificado invalido");
    return 0;
}
```

Console | I/O | AI Agent

```
Com o bug:  0 (certificado valido)
Corrigido: -1 (certificado invalido)
```

Observações: Durante esta execução simulamos um certificado com hash correto e assinatura falsa. A versão com o bug devolveu 0, indicando "certificado válido", pois o "goto fail" duplicado desviou a execução antes da verificação da assinatura. Na versão corrigida, com chaves em cada "if", a assinatura foi verificada e a função devolveu o erro de modo correto, rejeitando o certificado.

Slide 21:

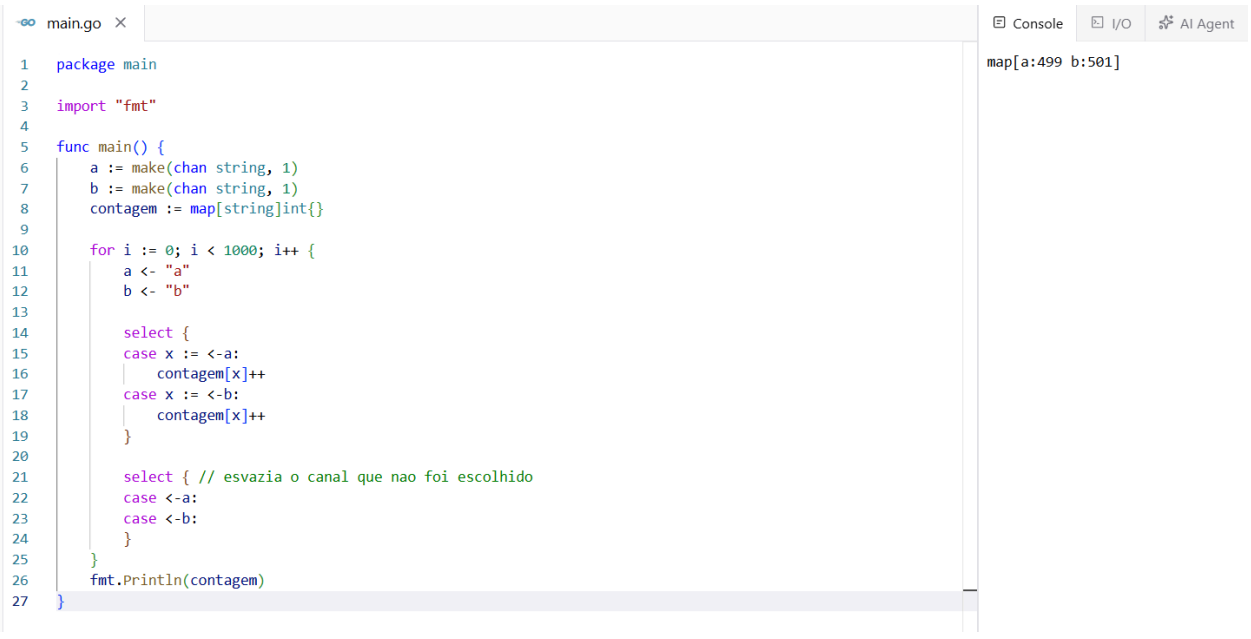
Comandos protegidos: Correspondem a estruturas de seleção e repetição propostas por Dijkstra, nas quais cada comando é protegido por uma condição, chamada de guarda, e só pode ser executado se

ela for verdadeira. Diferente do "if" frequente, em que as condições correspondem a testadas em ordem e a primeira verdadeira vence, no "if ... fi" todas as guardas correspondem a avaliadas. Se mais de uma for verdadeira, uma delas é escolhida de maneira não determinística, e se nenhuma for verdadeira, ocorre um erro de execução, o que obriga o programador a tratar todos os casos. No exemplo do máximo, quando x é igual a y, as duas guardas correspondem a verdadeiras, mas tanto faz qual ramo executa, pois o resultado é o mesmo. Assim, o código expressa apenas o que importa para o problema, sem impor uma ordem desnecessária entre os casos, o que facilita provar que o programa está correto.

O laço "do ... od" segue a mesma ideia: ele se repete enquanto alguma guarda for verdadeira, escolhendo uma ao acaso a cada volta, e se encerra quando nenhuma for. Com isso, o caminho percorrido pode alterar a cada execução, mas o desfecho continua garantido. Os comandos protegidos serviram de base para o CSP de Hoare (1978), e o "select" de Go é um descendente direto: quando vários canais estão prontos ao mesmo tempo, o Go sorteia qual deles será atendido. Já as guardas de Haskell herdaram o nome e a sintaxe, mas correspondem a avaliadas em ordem, sendo portanto determinísticas.

Evidências:

a) Go (select.go):
print 1ª execução:



print 2ª execução:

main.go

```go
package main

import "fmt"

func main() {
    a := make(chan string, 1)
    b := make(chan string, 1)
    contagem := map[string]int{}

    for i := 0; i < 1000; i++ {
        a <- "a"
        b <- "b"

        select {
        case x := <-a:
            contagem[x]++
        case x := <-b:
            contagem[x]++
        }

        select { // esvazia o canal que nao foi escolhido
        case <-a:
        case <-b:
        }
    }
    fmt.Println(contagem)
}
```

Console | I/O | AI Agent

```
map[a:489 b:511]
```

Observações: Nestas execuções, os dois canais estavam sempre prontos, e o "select" sorteou qual deles seria lido em cada uma das 1000 repetições. A contagem ficou próxima de metade para cada canal, mas os números mudaram de uma execução para a outra, mostrando a escolha não determinística entre guardas verdadeiras.

b) Python (guardas.py):

print 1ª execução:

main.py

```python
import random

q = [4, 3, 2, 1]
print("Inicio:", q)

while True:
    guardas = [i for i in range(3) if q[i] > q[i + 1]]
    if not guardas:
        break
    i = random.choice(guardas)
    q[i], q[i + 1] = q[i + 1], q[i]
    print(f"Guarda q{i+1} > q{i+2} escolhida: {q}")

print("Final:", q)
```

Console | I/O | AI Agent

```
Inicio: [4, 3, 2, 1]
Guarda q3 > q4 escolhida: [4, 3, 1, 2]
Guarda q2 > q3 escolhida: [4, 1, 3, 2]
Guarda q3 > q4 escolhida: [4, 1, 2, 3]
Guarda q1 > q2 escolhida: [1, 4, 2, 3]
Guarda q2 > q3 escolhida: [1, 2, 4, 3]
Guarda q3 > q4 escolhida: [1, 2, 3, 4]
Final: [1, 2, 3, 4]
```

print 2ª execução:

main.py

```python
import random

q = [4, 3, 2, 1]
print("Inicio:", q)

while True:
    guardas = [i for i in range(3) if q[i] > q[i + 1]]
    if not guardas:
        break
    i = random.choice(guardas)
    q[i], q[i + 1] = q[i + 1], q[i]
    print(f"Guarda q{i+1} > q{i+2} escolhida: {q}")

print("Final:", q)
```

Console | I/O | AI Agent

```
Inicio: [4, 3, 2, 1]
Guarda q3 > q4 escolhida: [4, 3, 1, 2]
Guarda q2 > q3 escolhida: [4, 1, 3, 2]
Guarda q1 > q2 escolhida: [1, 4, 3, 2]
Guarda q2 > q3 escolhida: [1, 3, 4, 2]
Guarda q3 > q4 escolhida: [1, 3, 2, 4]
Guarda q2 > q3 escolhida: [1, 2, 3, 4]
Final: [1, 2, 3, 4]
```

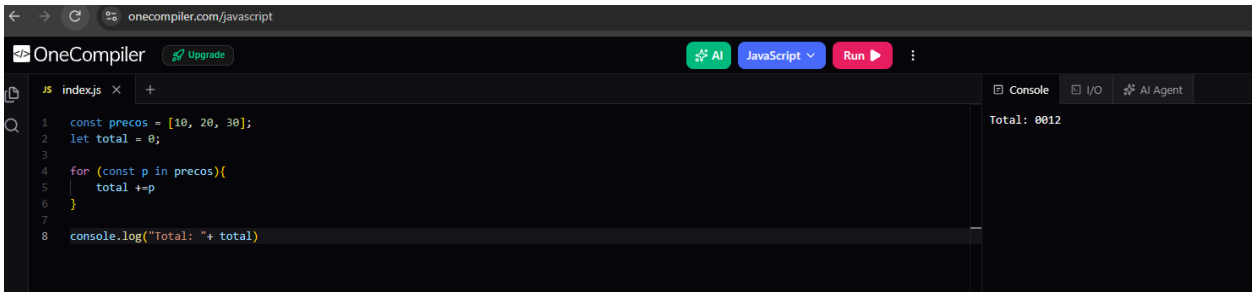
Observações: Nestas execuções, o programa simula o laço "do ... od" do livro, escolhendo ao

acaso uma das guardas verdadeiras (um par fora de ordem) e trocando seus valores. A sequência de trocas foi diferente em cada execução, mas o laço só terminou quando nenhuma guarda era verdadeira, e Por esse motivo o desfecho foi sempre a lista ordenada.

Slide 22:

JavaScript

A) Saida:



B) Causa do erro:

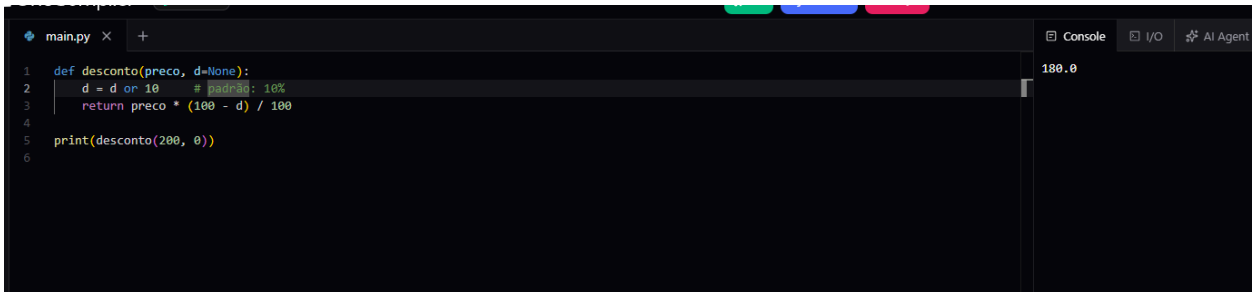
R: O erro é causado ao se utilizar o “in”, pois o in itera no total os índices de cada valor, não o seu valor de fato.

C) Como corrigir:

R: Para solucionar, é preciso primeiro substituir o operador `in` pelo of``, pois o ` irá acessar o valor de cada índice do array.

Python:

A) saida:



B) Causa do erro:

R: o erro é causado pelo operador “or”. Isso se deve ao fato de que, quando o or é usado em comparação entre dois valores e um deles é 0, o OR considera o 0 como uma expressão booleana. Desse modo, o código faz a leitura de que, se o valor for falso (0), então retorna 10. Desse modo, quando foram passados os valores 200 e 0, o sistema entendeu que era para puxar 10% de desconto.

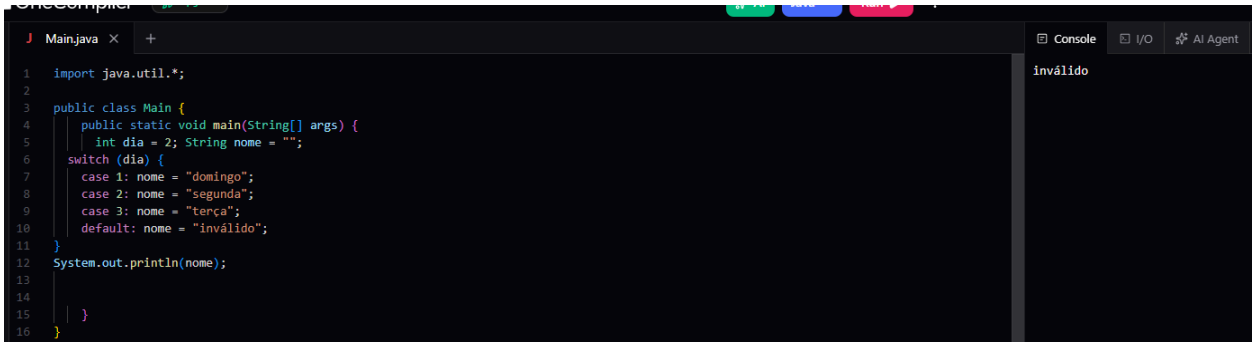
C) Como corrigir:

R: A verificação deve ser feita pelo is None`, como “if d is None”. Desse modo, quando se passa o valor 0, o sistema identifica que não é nulo, portanto vai passar 10, e se não passar nenhum valor, o sistema vai aplicar 10% de desconto.

Slide 23:

Java;

A) Saida:



B) Origem do erro:

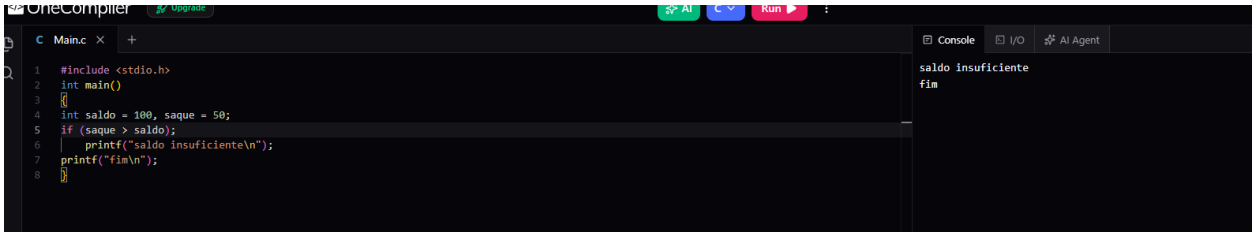
R: Origem do erro foi a falta de break a cada case, pois o sistema, sem o break, não sabe onde parar, Assim ele passa a executar todo o código, seguindo para então no default nome = "inválido", atribuindo esse valor final ao nome.

C) Como corrigir:

R: Para solucionar, basta informar o “break;” para cada case.

C;

A)Saida:



B) Origem do erro;

R: Origem do erro foi o uso da “;” depois do if, pois o saldo inválido não está dentro do if, porque o if se encerra antes de começar.

C) Como corrigir:

R: Para solucionar, é preciso retirar o “;” e adicionar algo dentro do if.

Slide 24:

Go:

A) Saida:



B) Origem do erro:

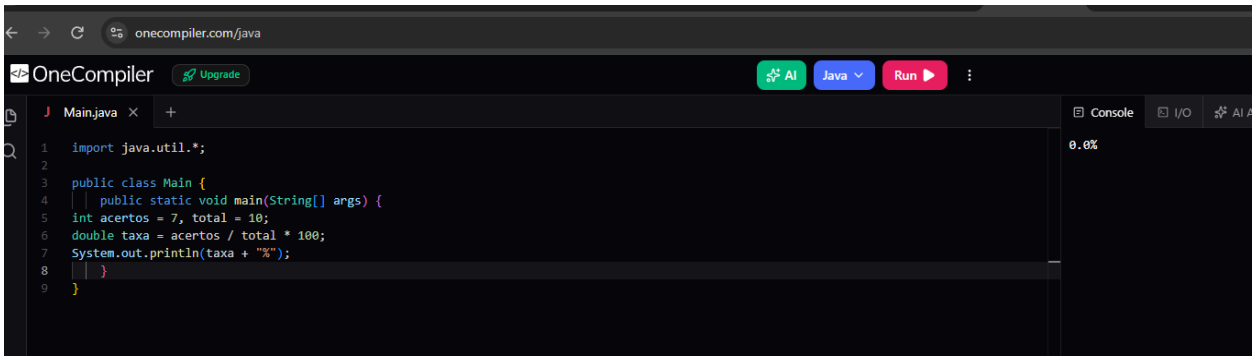
R: O operador := faz com que seja criada uma nova variável em vez de se utilizar a já existente.

C) Como corrigir;

R: Para solucionar, é suficiente alterar de := para =, ficando x = x * 2.

Java:

A) Saída:



B) Origem do erro:

R: Não apresenta o tratamento dos números inteiros antes de passar por uma contagem a partir de um double, Desse modo o sistema calcula 7/10 = 0, pois não apresenta decimais, e faz 0 x 100.

C) Como corrigir:

Para solucionar, é suficiente passar os acertos para double antes de começar a divisão.

```
double taxa = (double) acertos / total * 100;
```

Slide 25:

1) Precedência

A) Comportamento observado:

A conta -2 ** 2 dá -4 em vez de 4. Isso porque o sinal negativo é calculado depois da potência, então passa a ser $-(2^2) = -4$.

B) Motivo:
A regra de precedência. O operador ** (potência) apresenta prioridade maior que o - (negativo), então a potência é feita primeiro.

C) Como corrigir:
Colocar parênteses: (-2) ** 2. Assim o negativo é aplicado antes da potência.

2) Operadores

A) Comportamento observado:
A expressão +i + (+i) dá 0 em vez de incrementar. Porque + antes de parênteses não incrementa nada, é só um operador de positivo.

B) Motivo:
O + não é o incremento (++). Sem ++, a variável não muda, só soma o valor dela duas vezes.

C) Como corrigir:
Usar ++i ou i++ em vez de +i. Exemplo: i++; resultado = i + i;

3) Conversões

A) Comportamento observado:
A string "1.3" vira 1 (perde a parte decimal). A conversão automática trunca números decimais.

B) Motivo:
A conversão silenciosa de string para inteiro. A linguagem converte sem avisar e descarta os decimais.

C) Como corrigir:
Fazer a conversão explícita e correta. Em vez de atoi(), usar strtod() ou parseFloat() que preserva decimais.

4) Booleanos

A) Comportamento observado:
A string "0" é considerada verdadeira quando deveria ser falsa.

B) Motivo:
Em C e Python, strings não-vazias correspondem a sempre verdadeiras, mesmo que contenham "0". A linguagem não distingue a string "0" do número 0.

C) Como corrigir:
Ser explícito: fazer uma comparação clara em vez de converter direto. Exemplo: if (str != "") em vez de apenas if (str).

5) Atribuição

A) Comportamento observado:
if (x = y) atribui y em x em vez de comparar. Não testa igualdade, só coloca o valor.

B) Motivo:
= é atribuição, não comparação. É fácil confundir com == (que compara).

C) Como corrigir:
Usar == para comparar: if (x == y). Sempre usar == no if.

6) Seleção (Switch)

A) Comportamento observado:
Sem break, o switch executa vários cases seguidos ("fall-through"). Imprime mais coisas do que esperado.

B) Motivo:
Sem break, a execução não para após um case e continua pelos próximos.

C) Como corrigir:
Colocar break; ao final de cada case. Assim só executa o que você quer.

7) Laços

A) Comportamento observado:
O laço não incrementa e passa a ser infinito, ou nunca executa se a condição for falsa.

B) Motivo:
Falta de incremento (i++) ou inicialização errada no for.

C) Como corrigir:
Usar a sintaxe correta: for (int i = 1; i <= 10; i++). Sempre colocar a inicialização, condição e incremento.

8) Desvio

A) Comportamento observado:

goto ou return desvia o fluxo de maneira confusa e o código após não executa.

B) Motivo:

goto possibilita desvios aleatórios e return sai da função na hora. Passa a ser código desorganizado.

C) Como corrigir:

Evitar goto. Usar if/else, while, break, continue. Estruturas claras e ordenadas.

Slide 26:

A) O que é diferente:

Em Ruby, operadores como + e * parecem especiais, mas correspondem a na verdade métodos. Quando você escreve 1 + 2, Ruby traduz para 1.+(2). Em Lisp, tudo é de forma explícita uma função em notação prefixa: (+ 1 2).

B) Motivo a distinção:

Ruby quer parecer natural e legível como matemática frequente. Lisp prioriza simplicidade: sem precedência de operadores, os parênteses deixam claro o que fazer primeiro.

C) Como usar cada uma:

Ruby: escreva como de costume (1 + 2). Lisp: sempre prefixa (+ 1 2). Para somar vários números em Lisp, passa a ser (+ 1 2 3 4 5) em uma só função.

Conclusão: Ruby esconde que + é método para ser legível. Lisp expõe tudo. Ruby parece mais natural, Lisp é mais lógico.